UNIVERSIDADE FEDERAL DO RIO GRANDE – FURG
Centro de Ciências Computacionais – C3
Disciplina de Compiladores – Semestre 2026/2
Prof. Dr. Gerson Alberto Leiria Nunes

Trabalho Prático 1

Front-End de Compilador Pascal-S com Python PLY

Relatório técnico: decisões de projeto, gramática e análise de conflitos do parser

Integrantes: Manoela Rodrigues Sedrez e Sofia Botesini

Rio Grande, outubro de 2026

1 Introdução

Este relatório documenta o front-end de um compilador para o Pascal-S (Wirth, 1975), implementado em Python com a biblioteca PLY. A implementação parte dos requisitos do enunciado e cobre as construções centrais do Pascal-S: os tipos `integer`, `real`, `boolean` e `char`, constantes, tipos definidos pelo usuário, vetores, registros, funções e procedimentos (inclusive aninhados), as funções padrão, a entrada e saída com `read`/`write` e os comandos `if`, `while`, `for`, `repeat` e `case`. Além disso, o projeto mantém algumas extensões que *não* pertencem ao Pascal-S (ponteiros, o tipo `string`, a cláusula `uses` e o comentário `//`). As duas partes são descritas em seções separadas (seções 3 e 4), para deixar explícito o que é Pascal-S e o que vai além dele.

O front-end reúne análise léxica sob demanda, tabela de símbolos com ambientes encadeados, análise sintática LALR(1), verificação semântica e recuperação de erros sintáticos em modo pânico. As seções seguintes descrevem a arquitetura, a linguagem aceita, as decisões léxicas e semânticas, a gramática implementada, a análise do conflito shift/reduce observado em `parser.out`, os testes registrados e as limitações conhecidas.

2 Arquitetura

`Módulo`	Responsabilidade
`lex.py`	Tokens, palavras reservadas, comentários, aviso de `maxint`, linha/coluna, erros léxicos; expõe a fonte de tokens sob demanda (`getNextToken`).
`symtab.py`	Symbol, SymbolTable (ambientes encadeados por parent) e ScopeManager (pilha de ambientes).
`yacc.py`	Gramática LALR(1), ações semânticas (tipos e funções padrão), p_error com modo pânico, função run_file e linha de comando.
`run_tests.py`	Executa testes/*.pas e grava a saída real em logs/*.log.

Fluxo de dados. O parser é o único "motor": a cada vez que precisa de um lookahead ele invoca `tokenfunc=stream.getNextToken`, que delega a `lexer.getNextToken()` (apelido de `lexer.token()` do PLY). Nenhum vetor de tokens é construído: cada token é produzido, consumido e descartado. Ao produzir um token `ID`, o lexer chama `lookup_or_install` na tabela de símbolos e coloca o próprio objeto `Symbol` (uma referência) no atributo do token. O parser consulta e atualiza esse mesmo objeto nas ações semânticas, de modo que a tabela é, de fato, o repositório central de atributos.

Leitura do arquivo. O arquivo-fonte é lido de uma só vez para uma string (`read()`), porque a interface `lexer.input()` do PLY trabalha sobre uma cadeia. O que é preguiçoso é a *tokenização*: o texto só é varrido na medida em que o parser pede o próximo token. Uma consequência prática é que, se a análise é abortada, o restante do arquivo nunca é tokenizado (ver seção 8 e o teste `erros_multiplos.pas`, em que a linha 9 nunca chega a ser lida).

3 Pascal-S

Os recursos abaixo fazem parte do Pascal-S de Wirth e estão implementados e testados.

- **Tipos simples:** `integer`, `real`, `boolean` (com `true`/`false`) e `char`. O literal `'a'` (um único caractere) é do tipo `char`; `char` aceita os operadores relacionais e só é convertido de/para inteiro por `ord(c)` e `chr(i)`.
- **Constantes:** `const MAX = 10; PI = 3.14; LETRA = 'A'; ATIVO = true; NEG = -4;`, inclusive em subprogramas. A constante padrão `maxint` vale 2147483647.
- **Tipos estruturados:** `type`, vetores e matrizes (`array[1..3, 1..3] of real`) e registros (`record ... end`).
- **Subprogramas:** `function` e `procedure`, com parâmetros por valor e por referência (`var`), recursão e **subprogramas aninhados** em qualquer profundidade, cada um com seus próprios `const`, `type` e `var`.
- **Funções padrão:** `abs`, `sqr`, `sqrt`, `odd`, `succ`, `pred`, `round`, `trunc`, `sin`, `cos`, `exp`, `ln`, `arctan`, `ord` e `chr`, com checagem de tipo (tabela na seção 8).
- **Entrada e saída:** `write`, `writeln`, `read` e `readln`; `writeln` e `readln` também sem argumentos; largura de campo `x:w` e casas decimais `x:w:d` no `write`.
- **Comandos:** atribuição, `begin ... end`, `if`/`else`, `while`, `for`/`to`/`downto`, `repeat`/`until` e `case`/`of` (seletor `integer`, `char` ou `boolean`).
- **Operadores:** `+ - *`, `/` (divisão entre operandos do mesmo tipo numérico), `div`, `mod`, relacionais, `and`, `or`, `not` e sinal unário.
- **Comentários:** `(* ... *)`, o formato clássico do Pascal-S, e `{ ... }`.

4 Extensões fora do Pascal-S

Os recursos abaixo **não existem no Pascal-S**. Eles vêm do Pascal padrão ou do Turbo Pascal e foram mantidos como extensões do projeto, já implementados e testados.

- **Ponteiros (Pascal padrão):** `^tipo`, desreferência `p^` (encadeável: `p^.prox^.valor`), `nil`, `new(p)` e `dispose(p)`, com referência adiante (`PNo = ^No; No = record ... end`).
- **Tipo `string` (Turbo Pascal):** no Pascal-S, texto só aparece como literal dentro do `write`. Aqui `string` é um tipo declarável; `+` concatena e `=`/`<>` comparam. Um `char` pode ser atribuído a uma `string` e concatenado com ela (`s := 'letra: ' + c`).
- **Cláusula `uses` (Turbo Pascal):** registra os nomes das unidades e acusa repetição, mas não carrega bibliotecas reais.
- **Comentário `//`** (Delphi/Free Pascal).

5 Análise léxica

Tokens. Além das palavras reservadas básicas do enunciado (`program`, `var`, `integer`, `real`, `begin`, `end`, `if`, `then`, `else`, `while`, `do`, `writeln`, `not`, `and`, `or`), o analisador reconhece as palavras do Pascal-S `boolean`, `char`, `true`, `false`, `const`, `type`, `array`, `of`, `record`, `function`, `procedure`, `for`, `to`, `downto`, `repeat`, `until`, `case`, `div`, `mod`, `write`, `read` e `readln`, e as palavras das extensões `string`, `uses`, `nil`, `new` e `dispose`. Também são reconhecidos identificadores, números inteiros e reais, literais entre aspas, operadores e delimitadores. As funções padrão (`abs`, `sqrt`, `ord`, ...) e `maxint` não são palavras reservadas, e sim identificadores predefinidos: chegam ao parser como `ID` e são reconhecidos pelo nome na análise semântica.

Espaços em branco. Espaço, tabulação e retorno de carro (`t_ignore`: espaço, `\t` e `\r`) são descartados sem gerar token; a quebra de linha tem regra própria, que atualiza o contador de linhas.

Palavras reservadas. Não há uma expressão regular por palavra. O lexema é casado como identificador e depois filtrado em uma tabela hash (`reserved.get(valor.lower())`), o que também dá insensibilidade a maiúsculas/minúsculas (`BEGIN`, `Begin` e `begin` são o mesmo token).

Maior prefixo (longest match). Para tokens definidos por string, o PLY ordena as expressões pelo comprimento da regex, em ordem decrescente; assim `:=` vence `:` e `<=`/`<>` vencem `<`. Regras definidas por função são testadas antes das regras definidas por string, e entre elas vale a ordem de declaração: `NUM_REAL` é testado antes de `NUM_INT` (para que `3.14` não vire `3`, `.`, `14`) e os comentários são testados antes de `DIVIDE` e de `LPAREN` (para que `//` não seja duas divisões e `(*` não seja um parêntese seguido de multiplicação).

Comentários. São descartados sem retornar token: `(* ... *)` e `{ ... }`, que podem ocupar várias linhas (o contador de linhas é incrementado pelo número de quebras dentro do comentário), e `//` até o fim da linha. Um `(*` sem `*)` ou um `{` sem `}` correspondente gera erro léxico com linha e coluna, em vez de engolir silenciosamente o resto do arquivo.

Literais entre aspas. Um literal com exatamente um caractere (`'a'`) é do tipo `char`; os demais (`'texto'`, `''`) são `string`. Aspas duplicadas (`''''`) representam o próprio apóstrofo.

Aviso de `maxint`. Um literal inteiro maior que `maxint` (2147483647, o maior valor de um `integer` de 4 bytes) gera um *aviso* com linha e coluna. O aviso não conta como erro e não muda o resultado da compilação.

Linhas e colunas. A regra de quebra de linha incrementa `lexer.lineno`. A coluna é calculada por `find_column` a partir de `lexpos` e da última quebra de linha. Todo erro léxico ou sintático informa linha, coluna e o token ou caractere ofensor.

Erros léxicos. Um caractere fora do alfabeto (por exemplo `@`, `$`, `#`) é reportado e descartado (`lexer.skip(1)`), e a análise prossegue; os erros ficam registrados em `lexer.errors` para que o resultado final seja FALHA mesmo que o parser aceite o resto.

6 Tabela de símbolos

Entrada (Symbol). Cada identificador distinto corresponde a um único objeto com os atributos: `name` (lexema), `type` (o tipo declarado, ou `None` enquanto não declarado), `lineno` (linha da declaração ou do primeiro uso), `scope` (a `SymbolTable` em que vive), `offset` (deslocamento relativo no escopo), `declared` e `kind` (variável, constante, tipo, função ou procedimento). O offset é acumulado pela largura do tipo (`integer` = 4, `real` = 8, `boolean` = 1, `char` = 1; vetores e registros somam a largura dos elementos/campos).

Ponteiros. O valor do token `ID` é o objeto `Symbol`. Duas ocorrências do mesmo nome (inclusive com caixa diferente, pois a chave é `casefold()`) resultam no mesmo ponteiro.

Ambientes encadeados. `SymbolTable` é uma tabela hash com referência `parent` para o ambiente envolvente e campo `level`. O `lookup` sobe de filho para pai até o escopo global. `ScopeManager` mantém a pilha: `enter_scope` cria um filho e `exit_scope` volta ao pai (sem nunca passar do global).

Declarar e usar. O lexer instala o lexema ao vê-lo (`lookup_or_install`), mas a entrada nasce com `declared = False`. Somente a regra de declaração (`declare`) a marca como declarada. Assim: (a) usar um identificador cuja entrada continua não declarada é o erro variável não declarada; (b) declarar um nome que já está declarado no mesmo escopo é o erro redeclaração, informando a linha da primeira declaração; (c) declarar em um escopo interno um nome que só existe no externo é permitido e cria uma entrada local (sombreamento). As funções padrão e `maxint` só são reconhecidas quando o nome *não* foi declarado pelo programa; um subprograma do usuário chamado `abs`, por exemplo, tem prioridade.

Blocos e ambientes. Cada `begin ... end` abre um ambiente filho: a regra vazia `block_enter` (regra 64), reduzida logo após o `BEGIN`, chama `enter_scope`, e a redução de `block` chama `exit_scope`. O lexer continua sob demanda, e os nomes declarados no ambiente externo permanecem visíveis nos blocos internos pela cadeia `parent`. Os ramos de um `case` não abrem ambiente próprio: cada ramo é um comando, e só um ramo escrito como `begin ... end` cria o ambiente filho.

Subprogramas aninhados. Cada subprograma abre um ambiente filho do ambiente em que foi declarado, com seus parâmetros, constantes, tipos e variáveis locais. Um subprograma interno declarado dentro de outro vira filho do ambiente do externo, então enxerga os locais do externo pela cadeia `parent`, enquanto o externo não enxerga os locais do interno. No teste `sucesso_subprogramas_aninhados.pas`, a tabela de símbolos mostra quatro níveis: o global (nível 0), `externo` (nível 1), `soma` e `dobro` (nível 2) e `id`, declarada dentro de `dobro` (nível 3).

7 Gramática e parser LALR(1)

A gramática do enunciado foi usada como base, e o parser também reconhece o restante do Pascal-S e as extensões. A versão efetivamente carregada pelo PLY contém 156 produções numeradas de 0 a 155 em `parser.out` e gera 306 estados, numerados de 0 a 305. Portanto, a gramática efetiva é maior que a gramática mínima apresentada na especificação. A gramática completa está no apêndice B, primeiro em EBNF (visão geral)

e depois exatamente como foi passada ao PLY, com a numeração do `parser.out`; as seções a seguir comentam apenas as partes que exigiram decisões de projeto.

Na parte básica, as listas são construídas com recursão à esquerda, por exemplo `stmt_list : stmt_list SEMICOLON stmt`. A hierarquia `expr`, `simple_expr`, `term` e `factor` expressa os níveis de precedência dos operadores aritméticos e relacionais. As regras de recuperação usam o símbolo especial `error` do PLY.

Construções do Pascal-S além do mínimo. A gramática aceita `const`, `type`, vetores multidimensionais e registros; `for`/`to`/`downto`, `repeat`/`until` e `case`/`of` com `else` opcional; os operadores `div` e `mod` e o sinal unário; funções e procedimentos com parâmetros por valor e por referência (`var`), sendo os procedimentos chamados como comandos e as funções dentro de expressões; acesso a campos e indexação; e os comandos de entrada e saída. As funções padrão não precisam de produções próprias: usam a mesma regra de chamada `factor : ID LPAREN expr_list RPAREN` das funções do usuário e são distinguidas pelo nome na ação semântica.

Construções das extensões. A cláusula opcional `uses`, o tipo `string`, o tipo ponteiro `^tipo`, a desreferência `p^`, `nil` e os comandos `new` e `dispose`.

Subprogramas aninhados. A produção de subprograma repete a estrutura do programa principal, inclusive a lista de subprogramas, o que permite o aninhamento:

```
subprogram : sub_header const_opt type_opt decl_list subprogram_list block
   SEMICOLON
```

A ação de `sub_header` declara o nome no ambiente atual e abre o ambiente do subprograma; tudo o que vem depois (constantes, tipos, variáveis e subprogramas internos) é declarado nesse ambiente, e a redução de `subprogram` volta ao ambiente externo.

Entrada e saída. As regras 110 a 122 são:

```
write_stmt : WRITE LPAREN write_args RPAREN
           | WRITELN LPAREN write_args RPAREN
           | WRITELN
write_args : write_arg | write_args COMMA write_arg
write_arg  : expr | expr COLON expr | expr COLON expr COLON expr
read_stmt  : READ LPAREN read_args RPAREN
           | READLN LPAREN read_args RPAREN
           | READLN
read_args  : variable | read_args COMMA variable
```

Os argumentos de `read` usam o não-terminal `variable`, então a própria gramática já impede ler para dentro de uma expressão ou literal.

Comando case. As produções (regras 94 a 109) seguem a forma abaixo; o seletor é analisado em `case_head`, logo após o `of`, de modo que seu tipo já está disponível quando os rótulos de cada ramo são reduzidos.

```
case_stmt   : case_head case_list END
            | case_head case_list SEMICOLON END
            | case_head case_list case_else END
            | case_head case_list SEMICOLON case_else END
case_head   : CASE expr OF
case_else   : ELSE stmt | ELSE stmt SEMICOLON
```

```
case_list   : case_item | case_list SEMICOLON case_item
case_item   : case_labels stmt | error
case_labels : label_list COLON
label_list  : case_label | label_list COMMA case_label
case_label  : const_value
```

Como `stmt` pode ser vazio, um ramo vazio (`3:  ;`) e um `;` antes do `else` ou do `end` são aceitos, como em Pascal. Os rótulos reutilizam a regra `const_value` das declarações `const`, o que permite literais inteiros, inteiros negativos, caracteres (`'a'`), `true`/`false` e constantes nomeadas.

Precedência e associatividade. Os operadores são organizados pela estrutura das produções. Para o caso do *else pendente*, o parser declara `IFX` com precedência inferior a `ELSE`; a regra do `if` sem `else` recebe `%prec IFX`. Assim, diante de `ELSE`, o parser desloca o token e associa o `else` ao `if` mais próximo ainda sem alternativa.

8 Verificação semântica de tipos

As ações das regras sintáticas calculam o tipo sintetizado de cada expressão (avaliação dirigida por sintaxe, atributo sintetizado em pilha). Os tipos simples são `integer`, `real`, `boolean` e `char`, todos distintos; diferentemente do mínimo descrito no enunciado, a implementação permite declarar variáveis booleanas e usar as constantes `true` e `false`. Vetores, registros e, nas extensões, `string` e ponteiros também são verificados pelas regras específicas de suas construções. O valor `None` marca uma subexpressão que já gerou erro e é propagado em silêncio, evitando mensagens em cascata.

Construção	Regra aplicada	Erro quando
`+ - *`	Ambos os operandos do mesmo tipo numérico; resultado = esse tipo.	Tipos distintos (`integer` x `real`) ou operando não numérico.
`/`	Ambos os operandos do mesmo tipo numérico; resultado = esse tipo (`integer / integer` é `integer`; `real / real` é `real`).	Tipos distintos (`integer` x `real`) ou operando não numérico.
`div mod`	Ambos os operandos `integer`; resultado `integer`.	Algum operando não é inteiro.
`= <> < <= > >=`	Ambos do mesmo tipo numérico, ou ambos `char`; resultado boolean. Booleanos aceitam só `=` e `<>`.	Tipos distintos.
`and or`	Ambos boolean; resultado boolean.	Algum operando não é boolean.
`not`	Operando boolean; resultado boolean.	Operando não é boolean.
`x := e`	Tipo de `x` igual ao tipo de `e`.	Tipos diferentes (inclui real em integer, boolean em integer e integer em char).

`for / repeat`	Controle e limites de `for` são inteiros; condição de `repeat` é booleana.	Tipo incompatível.
`case e of`	Seletor ordinal (`integer`, `char` ou `boolean`); cada rótulo é uma constante do mesmo tipo do seletor e não se repete.	Seletor de outro tipo ou rótulo de tipo diferente (fatais); rótulo repetido ou não constante (não fatais).
`if / while`	Condição boolean.	Condição de outro tipo.
`write`	Valores escalares; largura `:w integer`; `:d` só para `real`.	Vetor, registro ou ponteiro inteiro; largura não inteira; `:d` em não real.
`read`	Variáveis declaradas do tipo `integer`, `real`, `char` (ou `string`).	Constante, variável boolean ou estruturada.
Chamada como comando	Só procedimentos; uma função (do usuário ou padrão) só aparece dentro de expressão.	Função usada como comando (`dobro(3);`, `sqrt(2.0);`).
Uso de ID	Entrada deve estar declarada (ou ser predefinida).	Variável não declarada.
Declaração	Nome novo no escopo atual.	Redeclaração no mesmo escopo.

Funções padrão. Cada função padrão recebe exatamente um argumento e tem o tipo do resultado definido pelo tipo do argumento:

Função	Argumento	Resultado
`abs`, `sqr`	`integer` ou `real`	mesmo tipo
`sqrt`, `sin`, `cos`, `exp`, `ln`, `arctan`	`integer` ou `real`	`real`
`odd`	`integer`	`boolean`
`succ`, `pred`	`integer`, `char` ou `boolean`	mesmo tipo
`round`, `trunc`	`real`	`integer`
`ord`	`integer`, `char` ou `boolean`	`integer`
`chr`	`integer`	`char`

Um argumento de tipo errado (por exemplo `round(7)` ou `odd(2.5)`) é erro de tipo fatal; um número errado de argumentos é registrado sem abortar.

Para as demais construções, a análise também verifica que índices de vetores sejam inteiros, que apenas vetores possam ser indexados, que os índices constantes respeitem os limites declarados (`maxint` também pode ser usado como limite), que campos acessados existam em registros, que campos não sejam declarados repetidamente e que argumentos de funções/procedimentos correspondam em quantidade e tipo aos parâmetros. Chamadas de procedimentos não são valores de expressão e, no sentido inverso, uma função não pode ser usada como comando, pois no Pascal-S o valor que ela devolve precisa ser consumido por uma expressão; funções devem respeitar o tipo de retorno. Nas extensões, ponteiros

são compatíveis com o mesmo tipo-alvo ou com `nil`, a alocação e a liberação exigem uma variável ponteiro, e a cláusula `uses` registra os nomes das unidades e acusa repetições.

No `case`, o seletor deve ser ordinal (`integer`, `char` ou `boolean`; `real` é rejeitado, pois não é enumerável). Ao entrar em um `case`, a ação de `case_head` empilha um contexto com o tipo do seletor e a tabela de rótulos já vistos; cada `case_labels` confere seus rótulos contra esse contexto e a redução de `case_stmt` desempilha. Assim, `case`s aninhados têm contextos independentes e um rótulo repetido é apontado com a linha da primeira ocorrência.

Sem conversões implícitas. Não existe conversão automática entre tipos. O operador `/` segue a mesma regra de `+ - *`: operandos do mesmo tipo numérico, com resultado desse tipo (a divisão `integer / integer` devolve `integer`, conforme a tipagem forte do enunciado); `div` e `mod` permanecem como operadores específicos de inteiros. A passagem de `real` para `integer` só acontece explicitamente, por `round` ou `trunc`, e a passagem entre `char` e `integer`, por `ord` e `chr`. As funções matemáticas (`sqrt`, `sin`, ...) aceitam argumento `integer`, como no Pascal-S original, e sempre devolvem `real`. Na extensão `string`, um `char` pode ser guardado em uma `string`, já que todo caractere é também um texto de tamanho 1.

Erros fatais e não fatais. Todos os erros de *tipo* da tabela (operandos incompatíveis, atribuição incompatível, `not`/`and`/`or` com operando não booleano, condição de `if`/`while` não booleana, seletor ou rótulo de `case` de tipo incompatível, argumento de função padrão e tipo inválido em `read`/`write`) são fatais, como pede o enunciado ("aborto imediato"): a ação semântica levanta a exceção `CompilationAbort`, que atravessa o PLY e é capturada em `run_file`, que imprime `Compilação abortada no primeiro erro de tipo`, despeja a tabela de símbolos e encerra com FALHA e código de saída 1. Já os erros de *declaração* (variável não declarada, em expressão ou em atribuição, e redeclaração), os de rótulo de `case` repetido ou não constante, os de número de argumentos e o de função usada como comando são registrados e a análise continua, de modo que vários deles podem ser listados em uma mesma execução. Como a tokenização é sob demanda, um erro fatal também impede que o restante do arquivo seja lido.

9 Recuperação de erros em modo pânico

O callback `p_error(p)` reporta a linha, a coluna e o token inesperado e em seguida entra em modo pânico: descarta tokens, obtidos sob demanda do mesmo fluxo preguiçoso, até encontrar um token de sincronização, que é devolvido ao fluxo (pushback) sem ser consumido. Ao retornar, o PLY insere o símbolo especial `error` como lookahead e desempilha estados até encontrar um que aceite `error`, usando as produções de recuperação previstas na gramática. A regra escolhida reduz, a análise segue a partir do sincronizador e novos erros podem ser reportados na mesma execução. O número de tokens descartados é informado no log (`[modo pânico] N token(s) descartado(s)`).

Desvio em relação ao enunciado. O enunciado adota apenas o ponto e vírgula como sincronizador. Aqui o conjunto é `{;, end, until}`. O motivo é gramatical: em Pascal o `;` separa comandos e o último comando de um bloco não o possui. Sem o `end` e o `until` no conjunto, um erro no último comando de um bloco ou de um `repeat` poderia engolir o delimitador e cascatear erros falsos (ou atravessar o programa até o fim do

arquivo). O teste `sync_end.pas` (`a :=` seguido de `end`) mostra exatamente esse caso: um único erro, no `end`, e a análise retomada no sincronizador. Quando o próprio token ofensor já é um sincronizador (`;`, `end` ou `until`), nada é descartado.

Garantia de progresso. Se, após a recuperação, o parser voltar a falhar exatamente no mesmo token (por exemplo um `else` solto, que não é sincronizador), esse token é descartado em silêncio, para evitar laço infinito. Esse caso é exercitado por `erro_else_solto`. No fim do arquivo (`p` nulo) o erro é reportado como `fim inesperado do arquivo` e não há recuperação possível.

Erros léxicos e erros sintáticos. Um caractere ilegal é descartado pelo lexer e não chega ao parser. Em `cont := 10 @ 5;` (cenário 2) o parser enxerga `10 5` e reporta também um erro sintático no `5`; os dois erros são reais e coexistem no log. Em seguida o parser se recupera no `;` e o `writeln` seguinte é analisado normalmente.

Recuperação dentro do case. A produção `case_item -> error` permite que um ramo malformado seja substituído por um item vazio: o erro é reportado, os tokens até o próximo `;` ou `end` são descartados e os ramos seguintes continuam sendo analisados. O teste `case_sintatico.pas` tem um ramo com expressão ausente (`1:  y := ;`) e outro com `:` faltando (`3 y := 3`); os dois são listados na mesma execução e o restante do `case` é aproveitado. Um erro no cabeçalho, antes do `of`, ainda não tem estado de recuperação próprio: o `end` do `case` serve de sincronizador e pode fechar o bloco envolvente mais cedo.

10 Análise de conflitos shift/reduce

Resultado global. A gramática foi gerada pelo PLY (LALR(1)) com debug habilitado. O único conflito registrado em `parser.out` é o do *else pendente*, resolvido por precedência (linha marcada com `!` no estado 247); não há outros conflitos registrados no arquivo gerado, inclusive após a inclusão do `case`, dos subprogramas aninhados e dos comandos de entrada e saída.

O conflito. As regras 86 (`if_stmt -> IF expr THEN stmt`) e 87 (`if_stmt -> IF expr THEN stmt ELSE stmt`) compartilham um prefixo. Depois de analisar o `stmt` do ramo `then`, o parser está em um estado que contém os dois itens LR(0):

```
state 247
    (86) if_stmt -> IF expr THEN stmt .
    (87) if_stmt -> IF expr THEN stmt . ELSE stmt

    END       reduce using rule 86 (if_stmt -> IF expr THEN stmt .)
    SEMICOLON reduce using rule 86 (if_stmt -> IF expr THEN stmt .)
    UNTIL     reduce using rule 86 (if_stmt -> IF expr THEN stmt .)
    ELSE      shift and go to state 284

  ! ELSE      [ reduce using rule 86 (if_stmt -> IF expr THEN stmt .) ]
```

Com lookahead `ELSE` há duas ações possíveis: reduzir pela regra 86 (fechar o `if` sem `else`) ou empilhar o `else` (regra 87, estado 284). Isso ocorre porque `ELSE` pertence ao conjunto FOLLOW de `if_stmt` (via `stmt`), já que um `if` aninhado no ramo `then` de outro pode ser seguido pelo `else` do externo. A gramática é, portanto, ambígua nesse ponto: `if a then if b then x else y` admite duas árvores.

Resolução. Foi declarada a precedência `IFX < ELSE` (ambos não associativos) e a regra 86 recebeu `%prec IFX`. Como o token `ELSE` tem precedência maior que a regra, o PLY resolve o conflito a favor do shift: cada `else` casa com o `if` mais próximo ainda sem `else` (a interpretação padrão de Pascal). A linha com `!` no estado 247 é a ação descartada. O teste `sucesso_else_pendente.pas` exercita exatamente essa construção.

Interação com o case. O `else` do `case` (regra 100) é outro uso do mesmo token, e por isso o `ELSE` passa a seguir também o último comando de um ramo. Isso não cria conflito novo: se o último comando do ramo termina em um `if` sem `else`, vale a resolução já descrita e o `else` pertence ao `if`; se termina em qualquer outro comando, o `else` só pode ser o do `case`. Para encerrar o `case` com o `else` de um ramo `if`, basta o `;` antes do `else` (`sucesso_case_if_else.pas`). Os rótulos também não competem com comandos, porque `case_labels` só é esperado logo após o `of` ou após um `;` dentro do `case`, e o `:` que os encerra não inicia nenhum comando.

Por que os demais casos não conflitam. (i) A produção vazia `stmt -> vazio` convive com o `if` sem conflito: no estado correspondente a `IF expr THEN . stmt` ela é reduzida apenas diante de `ELSE`, `END` e `;`, tokens que nenhum outro item do estado empilha. O comportamento da produção vazia depende do lookahead e é descrito pelas tabelas de estados geradas pelo PLY. A verificação de `parser.out` não mostra conflito adicional associado à produção vazia. (ii) Expressões: como a hierarquia `expr`, `simple_expr`, `term` e `factor` já separa os níveis, operadores de mesma classe usam recursão à esquerda (associatividade à esquerda) e operadores de classes diferentes não disputam o mesmo item. (iii) As produções com `error` usam o sincronizador seguinte (`decl_group -> error ';'`) ou aparecem como alternativa de `stmt` e de `case_item` e dependem de `stmt_list` ou `case_list` para o delimitador, então não criam ambiguidades novas. (iv) A regra vazia `block_enter` é a única ação possível logo após o `BEGIN`, portanto sua redução não compete com nenhum deslocamento. (v) `writeln` e `readln` sem argumentos não conflitam com as formas com argumentos: depois da palavra reservada, o lookahead `(` leva ao shift, e `;`, `end`, `else` ou `until` levam à redução. (vi) O `:` de `write(x:w)` só aparece dentro dos parênteses do `write`, onde nenhuma outra produção espera `:`. (vii) Nos subprogramas aninhados, as listas opcionais `const_opt`, `type_opt`, `decl_list` e `subprogram_list` começam por tokens distintos (`CONST`, `TYPE`, `VAR`, `FUNCTION`/`PROCEDURE`) e terminam antes do `BEGIN`, então cada redução vazia é decidida pelo lookahead.

Alternativa descartada. Reescrever a gramática com `matched_stmt` / `unmatched_stmt` eliminaria o conflito sem precedências, mas aumentaria o número de regras e se afastaria da gramática do enunciado; a precedência é a solução padrão em geradores LALR e deixa o parser mais próximo da especificação.

11 Casos de teste e resultados

O pacote contém 110 arquivos de entrada em `testes/` e os logs correspondentes em `logs/`. O script `python run_tests.py` executa a bateria e regrava os logs. A tabela abaixo apresenta uma seleção representativa dos cenários; os resultados são os registrados nos logs incluídos no pacote. Os três primeiros cenários correspondem aos casos obrigatórios do enunciado.

Teste	O que exercita	Resultado registrado
`sucesso_semantico.pas`	Cenário 1: tipos homogêneos, while, writeln.	SUCESSO (código 0).
`erro_lexico.pas`	Cenário 2: caractere ilegal '@' (5:14).	1 léxico + 1 sintático (5:16, efeito do descarte); FALHA.
`erro_tipo.pas`	Cenário 3: real atribuído a integer.	1 semântico (linha 8), compilação abortada; FALHA.
`erro_sintatico.pas`	Expressão ausente (`valor := ;`).	1 sintático (5:12), análise retomada; FALHA.
`erro_redeclaracao.pas`	Mesmo nome declarado duas vezes.	1 semântico (linha 4; 1ª declaração na linha 3); FALHA.
`erro_nao_declarada.pas`	Uso sem declaração.	1 semântico (linha 5), sem abortar; FALHA.
`erros_multiplos.pas`	Dois erros sintáticos e depois um erro de tipo.	Sintáticos em 6:8 e 7:12, semântico na linha 8 (aborta; a linha 9 não é lida); FALHA.
`recuperacao_semantica.pas`	Erro sintático seguido de erro de tipo.	1 sintático (5:16) + 1 semântico (linha 6); FALHA.
`erro_operador_duplo.pas`	Operador binário seguido de operador.	Sintáticos em 6:16 e 7:16, semântico na linha 8 (aborta); FALHA.
`erro_duas_na_mesma_linha.pas`	Dois erros sintáticos na mesma linha.	Sintáticos em 4:12 e 4:26, ambos recuperados; FALHA.
`erro_else_solto`	`else` sem `if`.	1 sintático (6:5), 3 tokens descartados; FALHA.
`sync_end.pas`	Erro no último comando do bloco.	1 sintático (6:1, no `end`), análise retomada; FALHA.
`erro_cabecalho.pas`	`program 123;`.	1 sintático (1:9); FALHA.
`erro_declaracao.pas`	`var x integer;` (dois-pontos ausente).	1 sintático (2:7); `y` é declarada normalmente; FALHA.
`erro_sem_end.pas`	Begin sem end.	1 sintático, fim inesperado do arquivo (6:1); FALHA.
`erro_sem_ponto.pas`	Ponto final ausente.	1 sintático, fim inesperado do arquivo (6:1); FALHA.
`erro_logicos_tipos.pas`	and/or/not com operandos integer.	1 semântico (linha 5, `and`), aborta; FALHA.
`erro_condicao_inteira.pas`	while/if com condição integer.	1 semântico (linha 5, `while`), aborta; FALHA.
`lex_extra.pas`	Palavras reservadas em caixas variadas, identificadores com _ e dígito, os três tipos de comentário, `$ # !` e comentário não fechado; o restante do programa é válido.	4 léxicos (`$`, `#` e `!` em 10:3, 10:5 e 10:7; `{` não fechado em 13:1), nenhum sintático ou semântico; FALHA.

Teste	O que exercita	Resultado registrado
`sucesso_else_pendente.pas`	If aninhado com um else (else pendente).	SUCESSO.
`sucesso_blocos_aninhados.pas`	Begin/end aninhados, reservadas em caixas variadas, comentário //.	SUCESSO.
`sucesso_stmt_vazio.pas`	Comandos vazios (`;;` e `;` antes do `end`).	SUCESSO.
`sucesso_case.pas`	Seletor integer e boolean, rótulos múltiplos, negativo e constante nomeada, ramo vazio, bloco, `else`, `case` aninhado.	SUCESSO.
`sucesso_case_if_else.pas`	`if` com `else` dentro de um ramo, seguido do `else` do `case`.	SUCESSO.
`erro_case_real.pas`	Seletor `real`.	1 semântico (linha 7), aborta; FALHA.
`erro_case_rotulo_tipo.pas`	Rótulo `true` com seletor integer.	1 semântico (linha 8), aborta; FALHA.
`erro_case_duplicado.pas`	Rótulo `2` repetido em ramos diferentes.	1 semântico (linha 9; 1ª ocorrência na linha 7), sem abortar; FALHA.
`erro_case_rotulo_variavel.pas`	Variável usada como rótulo.	1 semântico (linha 8), sem abortar; FALHA.
`case_sintatico.pas`	Expressão ausente em um ramo e `:` ausente em outro.	Sintáticos em 7:13 e 9:7, ambos recuperados; FALHA.
Recursos do Pascal-S acrescentados nesta versão		
`sucesso_char.pas`	`char`, constante `'A'`, `ord`/`chr`, relacionais, `case` com rótulos `char`, `char` em `string`.	SUCESSO.
`sucesso_funcoes_padrao.pas`	As 13 funções numéricas e ordinais (`ord`/`chr` estão em `sucesso_char`) e `maxint`.	SUCESSO.
`sucesso_read_write.pas`	`read`, `readln` (com e sem argumentos, inclusive `v[2]`), `write`, `writeln` sem argumentos, `n:5` e `x:8:2`.	SUCESSO.
`sucesso_comentario_classico.pas`	`(* ... *)` em várias linhas, junto com `{ }` e `//`.	SUCESSO.
`sucesso_subprogramas_aninhados.pas`	Procedimento e função internos, `const`/`type` locais, acesso a locais do externo, três níveis de aninhamento.	SUCESSO; tabela com escopos de nível 0 a 3.
`aviso_maxint.pas`	Literal `3000000000`, maior que `maxint`.	Aviso (6:8), sem erro; SUCESSO.

Teste	O que exercita	Resultado registrado
`sucesso_div_inteira.pas`	`a / 2` com inteiros e `x / 2.0` com reais.	SUCESSO.
`erro_div_misto.pas`	`a / x` com `integer` e `real`.	1 semântico (linha 8), aborta; FALHA.
`erro_div_boolean.pas`	`p / p` com booleanos.	1 semântico (linha 6), aborta; FALHA.
`erro_char_inteiro.pas`	Inteiro atribuído a `char`.	1 semântico (linha 5), aborta; FALHA.
`erro_case_char_rotulo.pas`	Rótulo inteiro com seletor `char`.	1 semântico (linha 8), aborta; FALHA.
`erro_funcao_padrao_tipo.pas`	`round(7)`.	1 semântico (linha 5), aborta; FALHA.
`erro_odd_real.pas`	`odd(2.5)`.	1 semântico (linha 5), aborta; FALHA.
`erro_funcao_padrao_args.pas`	`abs(1, 2)`.	1 semântico (linha 5), sem abortar; FALHA.
`erro_read_boolean.pas`	`read` de variável boolean.	1 semântico (linha 5), aborta; FALHA.
`erro_read_constante.pas`	`readln(MAX)` com `MAX` constante.	1 semântico (linha 5), sem abortar; FALHA.
`erro_write_largura.pas`	Largura real (`n:2.0`).	1 semântico (linha 6), aborta; FALHA.
`erro_write_decimais.pas`	Casas decimais em inteiro (`n:5:2`).	1 semântico (linha 6), aborta; FALHA.
`erro_aninhado_escopo.pas`	Externo usando variável local do interno.	1 semântico (linha 14), sem abortar; FALHA.
`erro_comentario_classico_aberto.pas`	`(*` sem `*)`.	1 léxico (5:11) + fim inesperado do arquivo (8:1); FALHA.
`erro_funcao_como_comando.pas`	Função do usuário (`dobro(3);`) e função padrão (`sqrt(2.0);`) usadas como comando.	2 semânticos (linhas 11 e 12), sem abortar; FALHA.

Exemplo - cenário 3 (log completo, `erro_tipo`):

```
Erro semântico (linha 8): atribuição incompatível: 'cont' é integer, mas a
    expressão é real.
Compilação abortada no primeiro erro de tipo.

TABELA DE SIMBOLOS
[escopo 'global' nivel 0]
  nome              tipo     offset  linha
  cont              integer  0       3
  acumulador        real     4       4
  taxa              real     12      4
```

```
Erros léxicos: 0
Erros sintáticos: 0
Erros semânticos: 1

Resultado: FALHA
```

Exemplo - vários erros em uma execução (`erros_multiplos`):

```
Erro sintático [linha 6, coluna 8]: token inesperado SEMICOLON (';').
Recuperação sintática: análise retomada no sincronizador.
Erro sintático [linha 7, coluna 12]: token inesperado TIMES ('*').
    [modo pânico] 1 token(s) descartado(s).
Recuperação sintática: análise retomada no sincronizador.
Erro semântico (linha 8): atribuição incompatível: 'x' é real, mas a expressão é
    integer.
Compilação abortada no primeiro erro de tipo.
```

Exemplo - recuperação dentro do case (`case_sintatico`):

```
Erro sintático [linha 7, coluna 13]: token inesperado SEMICOLON (';').
Recuperação sintática: análise retomada no sincronizador.
Erro sintático [linha 9, coluna 7]: token inesperado ID ('y').
    [modo pânico] 2 token(s) descartado(s).
Recuperação sintática: análise retomada no sincronizador.
```

Exemplo - subprogramas aninhados (`sucesso_subprogramas_aninhados`, trecho da tabela):

```
[escopo 'proc externo' nivel 1]
  nome              tipo                        offset  linha
  n                 integer                     0       5
  PASSO             const integer = 2           -       7
  Par               type = Par                  -       9
  acc               integer                     4       11
  p                 Par                         8       12
  soma              procedure(integer)          -       14
  dobro             function(integer): integer  -       19
[escopo 'proc soma' nivel 2]
  nome              tipo     offset  linha
  k                 integer  0       14
[escopo 'func dobro' nivel 2]
  nome              tipo                        offset  linha
  k                 integer                     0       19
  id                function(integer): integer  -       20
[escopo 'func id' nivel 3]
  nome              tipo     offset  linha
  m                 integer  0       20
```

Exemplo - divisão com tipos distintos (`erro_div_misto`):

```
Erro semântico (linha 8): operandos incompatíveis para '/' (integer e real); não
    há conversão implícita entre tipos distintos.
Compilação abortada no primeiro erro de tipo.
```

12 Limitações e conclusão

Limitações conhecidas. (i) A cláusula `uses` (extensão) registra os nomes das unidades, mas não implementa nem carrega as bibliotecas correspondentes. (ii) A linguagem implementada cobre o Pascal-S, mas não é um compilador Pascal completo: não há tipos enumerados nem subintervalo, conjuntos (`set`), arquivos, `goto`/`label`, `with`, nem as funções `eof` e `eoln`. (iii) Erros semânticos de tipo são fatais, então os erros posteriores no arquivo não são necessariamente reportados. (iv) Um caractere ilegal é descartado pelo lexer e pode provocar também um erro sintático consequente. (v) O arquivo é carregado em uma string, conforme a interface do PLY, mas a tokenização ocorre sob demanda. (vi) No `case`, os rótulos são constantes isoladas, sem intervalos como `1..5`, e a palavra `otherwise` não é aceita no lugar de `else`. (vii) O valor de `maxint` (2147483647) segue o `integer` de 4 bytes da tabela de símbolos, e não a palavra de 48 bits da máquina CDC usada por Wirth; um literal maior gera apenas aviso. (viii) A variável de controle do `for` precisa ser `integer`; `char` e `boolean` não são aceitos nessa posição.

Conclusão. A implementação contempla os principais requisitos do TP1: tokenização sob demanda com informação de linha e coluna, tabela de símbolos com referências a objetos e ambientes encadeados, parser LALR(1), verificação sem conversões implícitas e recuperação sintática em modo pânico. O conflito do *else pendente* foi identificado no estado 247 do `parser.out` e resolvido por precedência. Do Pascal-S, o projeto implementa também `char`, `const`, `type`, vetores, matrizes, registros, funções e procedimentos aninhados com parâmetros por valor e `var`, as funções padrão, `read`/`write`, `for`, `repeat`, `case`, `div`, `mod`, o operador `/` e o comentário `(* *)`. Fora do Pascal-S, mantém como extensões os ponteiros, o tipo `string`, a cláusula `uses` e o comentário `//`. O pacote contém testes e seus logs correspondentes; os resultados registrados devem ser interpretados conforme cada caso, distinguindo programas válidos dos testes que verificam a detecção de erros.

A Execução

```
python3 -m pip install ply==3.11
python3 yacc.py testes/sucesso_semantico.pas
python3 lex.py testes/lex_extra.pas
python3 run_tests.py
```

B Gramática completa

Visão geral em EBNF. A notação segue a do enunciado: `[ ]` indica parte opcional, `{ }` repetição de zero ou mais vezes e `( | )` alternativa. As palavras entre aspas são palavras reservadas ou símbolos; `ID`, `NUM_INT`, `NUM_REAL` e `STRING` são as classes de tokens do analisador léxico. As construções marcadas como extensão não pertencem ao Pascal-S (seção 4). As produções de recuperação de erro e a regra auxiliar `block_enter` aparecem apenas na listagem seguinte.

```
program      ::= "program" ID ";" [ uses ] [ const_part ] [ type_part ] [ var_part ]
                 { subprogram } block "."
uses         ::= "uses" ID { "," ID } ";"                              (* extensão *)
```

```
const_part   ::= "const" const_decl { const_decl }
const_decl   ::= ID "=" const_value ";"
const_value  ::= [ "-" ] ( NUM_INT | NUM_REAL ) | STRING | "true" | "false" | ID
type_part    ::= "type" type_decl { type_decl }
type_decl    ::= ID "=" type ";"
type         ::= "integer" | "real" | "boolean" | "char" | ID
               | "array" "[" range { "," range } "]" "of" type
               | "record" field_decl { ";" field_decl } [ ";" ] "end"
               | "string" | "^" type                                  (* extensões *)
range        ::= bound ".." bound
bound        ::= [ "-" ] NUM_INT | ID
field_decl   ::= id_list ":" type
var_part     ::= "var" decl_group { decl_group }
decl_group   ::= id_list ":" type ";"
id_list      ::= ID { "," ID }
subprogram   ::= sub_header [ const_part ] [ type_part ] [ var_part ]
                 { subprogram } block ";"
sub_header   ::= "function" ID [ "(" params ")" ] ":" type ";"
               | "procedure" ID [ "(" params ")" ] ";"
params       ::= param_group { ";" param_group }
param_group  ::= [ "var" ] id_list ":" type
block        ::= "begin" stmt_list "end"
stmt_list    ::= stmt { ";" stmt }
stmt         ::= variable ":=" expr
               | ID [ "(" [ expr_list ] ")" ]                         (* chamada *)
               | if_stmt | while_stmt | for_stmt | repeat_stmt | case_stmt
               | write_stmt | read_stmt | block
               | ( "new" | "dispose" ) "(" variable ")"               (* extensão *)
               | empty
if_stmt      ::= "if" expr "then" stmt [ "else" stmt ]
while_stmt   ::= "while" expr "do" stmt
for_stmt     ::= "for" ID ":=" expr ( "to" | "downto" ) expr "do" stmt
repeat_stmt  ::= "repeat" stmt_list "until" expr
case_stmt    ::= "case" expr "of" case_item { ";" case_item } [ ";" ]
                 [ "else" stmt [ ";" ] ] "end"
case_item    ::= const_value { "," const_value } ":" stmt
write_stmt   ::= ( "write" | "writeln" ) "(" write_arg { "," write_arg } ")"
               | "writeln"
write_arg    ::= expr [ ":" expr [ ":" expr ] ]
read_stmt    ::= ( "read" | "readln" ) "(" variable { "," variable } ")"
               | "readln"
variable     ::= ID { "." ID | "[" expr_list "]" | "^" }
expr_list    ::= expr { "," expr }
expr         ::= simple_expr [ relop simple_expr ]
simple_expr  ::= [ "+" | "-" ] term { addop term }
term         ::= factor { mulop factor }
factor       ::= variable | ID "(" [ expr_list ] ")" | NUM_INT | NUM_REAL | STRING
               | "true" | "false" | "nil" | "(" expr ")" | "not" factor
relop        ::= "=" | "<>" | "<" | "<=" | ">" | ">="
addop        ::= "+" | "-" | "or"
mulop        ::= "*" | "/" | "div" | "mod" | "and"
empty        ::=
```

Produções exatamente como foram passadas ao PLY. Cada repetição `{ }` da EBNF vira uma regra recursiva à esquerda e cada parte opcional vira uma regra com alternativa vazia (`(vazio)`). Os números entre parênteses são os do `parser.out` (a regra 0, `S' -> program`, é criada pelo próprio PLY). As produções estão agrupadas

pelo não-terminal da esquerda, por isso a numeração de um mesmo grupo nem sempre é contínua. Os terminais são os nomes de tokens de `lex.py`: por exemplo, `SEMICOLON` é `;`, `ASSIGN` é `:=`, `DOTDOT` é `..`, `CARET` é `^`, `STRINGTYPE` é a palavra `string` e `CHARTYPE` é a palavra `char`.

```
 (1) program         : PROGRAM ID SEMICOLON uses_opt const_opt type_opt decl_list
  subprogram_list block DOT
 (2)                 | PROGRAM error SEMICOLON uses_opt const_opt type_opt decl_list
  subprogram_list block DOT

 (3) uses_opt        : (vazio)
 (4)                 | USES unit_list SEMICOLON

 (5) unit_list       : ID
 (6)                 | unit_list COMMA ID

 (7) const_opt       : (vazio)
 (8)                 | CONST const_decl_list

 (9) const_decl_list : const_decl
(10)                 | const_decl_list const_decl

(11) const_decl      : ID EQ const_value SEMICOLON

(12) const_value     : NUM_INT
(13)                 | NUM_REAL
(14)                 | STRING
(15)                 | TRUE
(16)                 | FALSE
(17)                 | MINUS NUM_INT
(18)                 | MINUS NUM_REAL
(19)                 | ID

(20) type_opt        : (vazio)
(21)                 | TYPE type_decl_list

(22) type_decl_list  : type_decl
(23)                 | type_decl_list type_decl

(24) type_decl       : ID EQ type SEMICOLON

(25) type            : ID
(27)                 | ptr_start type
(28)                 | STRINGTYPE
(29)                 | RECORD field_decl_list END
(30)                 | RECORD field_decl_list SEMICOLON END
(53)                 | INTEGER
(54)                 | BOOLEAN
(55)                 | CHARTYPE
(56)                 | ARRAY LBRACKET range_list RBRACKET OF type
(63)                 | REAL

(26) ptr_start       : CARET

(31) field_decl_list : field_decl
(32)                 | field_decl_list SEMICOLON field_decl

(33) field_decl      : id_list COLON type
```

```
(34) subprogram_list : (vazio)
(35)                 | subprogram_list subprogram

(36) subprogram      : sub_header const_opt type_opt decl_list subprogram_list block
   SEMICOLON

(37) sub_header      : FUNCTION ID LPAREN params RPAREN COLON type SEMICOLON
(38)                 | FUNCTION ID COLON type SEMICOLON
(39)                 | PROCEDURE ID LPAREN params RPAREN SEMICOLON
(40)                 | PROCEDURE ID SEMICOLON

(41) params          : param_group
(42)                 | params SEMICOLON param_group

(43) param_group     : id_list COLON type
(44)                 | VAR id_list COLON type

(45) decl_list       : (vazio)
(46)                 | VAR decl_group_list

(47) decl_group_list : decl_group
(48)                 | decl_group_list decl_group

(49) decl_group      : id_list COLON type SEMICOLON
(50)                 | error SEMICOLON

(51) id_list         : ID
(52)                 | id_list COMMA ID

(57) range_list      : range
(58)                 | range_list COMMA range

(59) range           : bound DOTDOT bound

(60) bound           : NUM_INT
(61)                 | ID
(62)                 | MINUS NUM_INT

(64) block_enter     : (vazio)

(65) block           : BEGIN block_enter stmt_list END

(66) stmt_list       : stmt
(67)                 | stmt_list SEMICOLON stmt

(68) stmt            : NEW LPAREN variable RPAREN
(69)                 | DISPOSE LPAREN variable RPAREN
(70)                 | ID LPAREN expr_list RPAREN
(71)                 | ID LPAREN RPAREN
(72)                 | variable
(73)                 | assign_stmt
(74)                 | if_stmt
(75)                 | while_stmt
(76)                 | write_stmt
(77)                 | read_stmt
(78)                 | block
(79)                 | (vazio)
```

```
 (80)                | error
 (89)                | for_stmt
 (90)                | repeat_stmt
 (95)                | case_stmt

 (81) assign_stmt    : variable ASSIGN expr

 (82) variable       : ID
 (83)                | variable DOT ID
 (84)                | variable CARET
 (85)                | variable LBRACKET expr_list RBRACKET

 (86) if_stmt        : IF expr THEN stmt
 (87)                | IF expr THEN stmt ELSE stmt

 (88) while_stmt     : WHILE expr DO stmt

 (91) for_stmt       : FOR ID ASSIGN expr TO expr DO stmt
 (92)                | FOR ID ASSIGN expr DOWNTO expr DO stmt

 (93) repeat_stmt    : REPEAT stmt_list UNTIL expr

 (94) case_head      : CASE expr OF

 (96) case_stmt      : case_head case_list END
 (97)                | case_head case_list SEMICOLON END
 (98)                | case_head case_list case_else END
 (99)                | case_head case_list SEMICOLON case_else END

(100) case_else      : ELSE stmt
(101)                | ELSE stmt SEMICOLON

(102) case_list      : case_item
(103)                | case_list SEMICOLON case_item

(104) case_item      : case_labels stmt
(105)                | error

(106) case_labels    : label_list COLON

(107) label_list     : case_label
(108)                | label_list COMMA case_label

(109) case_label     : const_value

(110) write_stmt     : WRITE LPAREN write_args RPAREN
(111)                | WRITELN LPAREN write_args RPAREN
(112)                | WRITELN

(113) write_args     : write_arg
(114)                | write_args COMMA write_arg

(115) write_arg      : expr
(116)                | expr COLON expr
(117)                | expr COLON expr COLON expr

(118) read_stmt      : READ LPAREN read_args RPAREN
(119)                | READLN LPAREN read_args RPAREN
```

```
(120)                 | READLN

(121) read_args       : variable
(122)                 | read_args COMMA variable

(123) expr_list       : expr
(124)                 | expr_list COMMA expr

(125) expr            : simple_expr
(126)                 | simple_expr relop simple_expr

(127) relop           : EQ
(128)                 | NE
(129)                 | LT
(130)                 | LE
(131)                 | GT
(132)                 | GE

(133) simple_expr     : term
(134)                 | PLUS term
(135)                 | MINUS term
(136)                 | simple_expr PLUS term
(137)                 | simple_expr MINUS term
(138)                 | simple_expr OR term

(139) term            : factor
(140)                 | term TIMES factor
(141)                 | term DIVIDE factor
(142)                 | term DIV factor
(143)                 | term MOD factor
(144)                 | term AND factor

(145) factor          : variable
(146)                 | ID LPAREN expr_list RPAREN
(147)                 | ID LPAREN RPAREN
(148)                 | NIL
(149)                 | STRING
(150)                 | NUM_INT
(151)                 | NUM_REAL
(152)                 | TRUE
(153)                 | FALSE
(154)                 | LPAREN expr RPAREN
(155)                 | NOT factor
```